Solutions

FDE Tutorial Sessions, Week 3

Exercise 1

Task 1.1*

```
join -t'|' -1 3 -2 1 \
```

join field 3 of nation (n_regionkey) with field 1 of region (r_regionkey)

```
  <(sort -t'|' -k3 nation.tbl) \
```

join needs its **input sorted** on the **join field**

```
  <(grep EUROPE region.tbl) \
```

already sorted by continent name

```
| cut -d '|' -f3
```

join output: 1 = key, 2 = n_nationkey, **3 = n_name**, …

Result:

```
UNITED KINGDOM
GERMANY
FRANCE
RUSSIA
ROMANIA
```

Task 1.1* - alternative solution

```
join -t'|' -1 3 -2 1 <(sort -t'|' -k3 nation.tbl) region.tbl \
  | grep EUROPE | cut -d'|' -f3
```

Task 1.1* - alternative solution

```
join -t'|' -1 3 -2 1 <(sort -t'|' -k3 nation.tbl) region.tbl \
  | grep EUROPE | cut -d'|' -f3
```

OPTION 1: select → join

join outputs 5 rows

grep reads 5 region rows

OPTION 2: join → select

join outputs all 25 nations

grep reads all of them

Takeaways: join needs sorted input, and **filter as early as possible** (less data goes through the rest of the processing → more efficient!)

Task 1.2

```
$ wc -l lineitem.tbl
6001215 lineitem.tbl
```

Task 1.3*

All rows of the same part are next to each other

awk adds up quantities until the key changes → only one group in memory at a time

```
sort -n -t'|' -k2 lineitem.tbl |
awk -F '|' '
  BEGIN { partkey = 0; sum_quantity = 0 }
  {
    if (partkey != $2) {
      print partkey, sum_quantity
      partkey = $2
      sum_quantity = 0
    }
    sum_quantity += $5
  }
  END { print partkey, sum_quantity }
'
```



Task 1.3* - alternative solution

awk keeps a running sum per part key in an array and **reads the file once**, without sorting

```
awk -F'|' '
  { groups[$2] += $5 }       # sum per l_partkey
  END {
    for (partkey in groups)
      print partkey, groups[partkey]
  }
' lineitem.tbl
```

A missing key in an awk array starts at 0, no initialisation is needed.

Task 1.4

	sort first	hash table
speed	slower: sorting is O(n log n)	usually faster: one pass, O(n)
memory	one group at a time	all groups at once
too much data	still works: sort saves data to temporary files and merges them	fails when the groups no longer fit into memory

```
real    0m9.046s
user    0m33.772s
sys     0m3.793s
```

```
real    0m3.846s
user    0m3.187s
sys     0m0.656s
```

Task 1.5

	sort first	hash table
	real 0m9.046s user 0m33.772s sys 0m3.793s	real 0m3.846s user 0m3.187s sys 0m0.656s
after `> /dev/null`	real 0m5.923s user 0m30.585s sys 0m1.590s	real 0m3.623s user 0m3.455s sys 0m0.168s

Task 1.6

	sort first	hash table
	real 0m9.046s user 0m33.772s sys 0m3.793s	real 0m3.846s user 0m3.187s sys 0m0.656s
after `> /dev/null`	real 0m5.923s user 0m30.585s sys 0m1.590s	real 0m3.623s user 0m3.455s sys 0m0.168s
after `| head`	real 0m3.142s user 0m24.813s sys 0m1.011s	real 0m3.651s user 0m3.272s sys 0m0.368s

Exercise 2

Task 2.1*

```
join -t'|' \
```

join on field 1 of both inputs (the default)

```
  <(grep -E '^([^|]*\|){5}1-URGENT\|' orders.tbl | cut -d'|' -f1 | sort) \
```

urgent orders only (next slide) → keep o_orderkey → sort as text

```
  <(cut -d'|' -f1,5 lineitem.tbl | sort -t'|' -k1,1) \
```

keep l_orderkey and l_quantity → sort by the key field **only**

```
| awk -F'|' '{sum += $2} END {print sum}'
```

join output is orderkey|quantity → add up field 2

Task 2.1*

`^`	`([^|]*|){5}`	`1-URGENT`	`|`
start of line	5 × (any characters except | , then a |) **= skip fields 1–5**	**field 6 = o_orderpriority**	end of the field

```
42|1234|O|98765.43|1995-03-14|1-URGENT|Clerk#000000123|0|…|
```

Task 2.2

```
double sum = 0;
for (const Lineitem& lineitem : lineitems) {
    for (const Order& order : orders) {
        if (lineitem.orderkey == order.orderkey && order.orderpriority == "1-URGENT") {
            sum += lineitem.quantity;
        }
    }
}
return sum;
```

Every line item is compared with every order:

|lineitem| · **|orders|** pairs

Nested Loop Join

Compare **every** row of R with **every** row of S

```
for (const auto& r : R)
    for (const auto& s : S)
        if (r.key == s.key)
            emit(r, s);
```

- Cost: **|R| · |S|** comparisons → O(n·m)
- Double both inputs → **4×** the comparisons

\+ simple, no extra memory

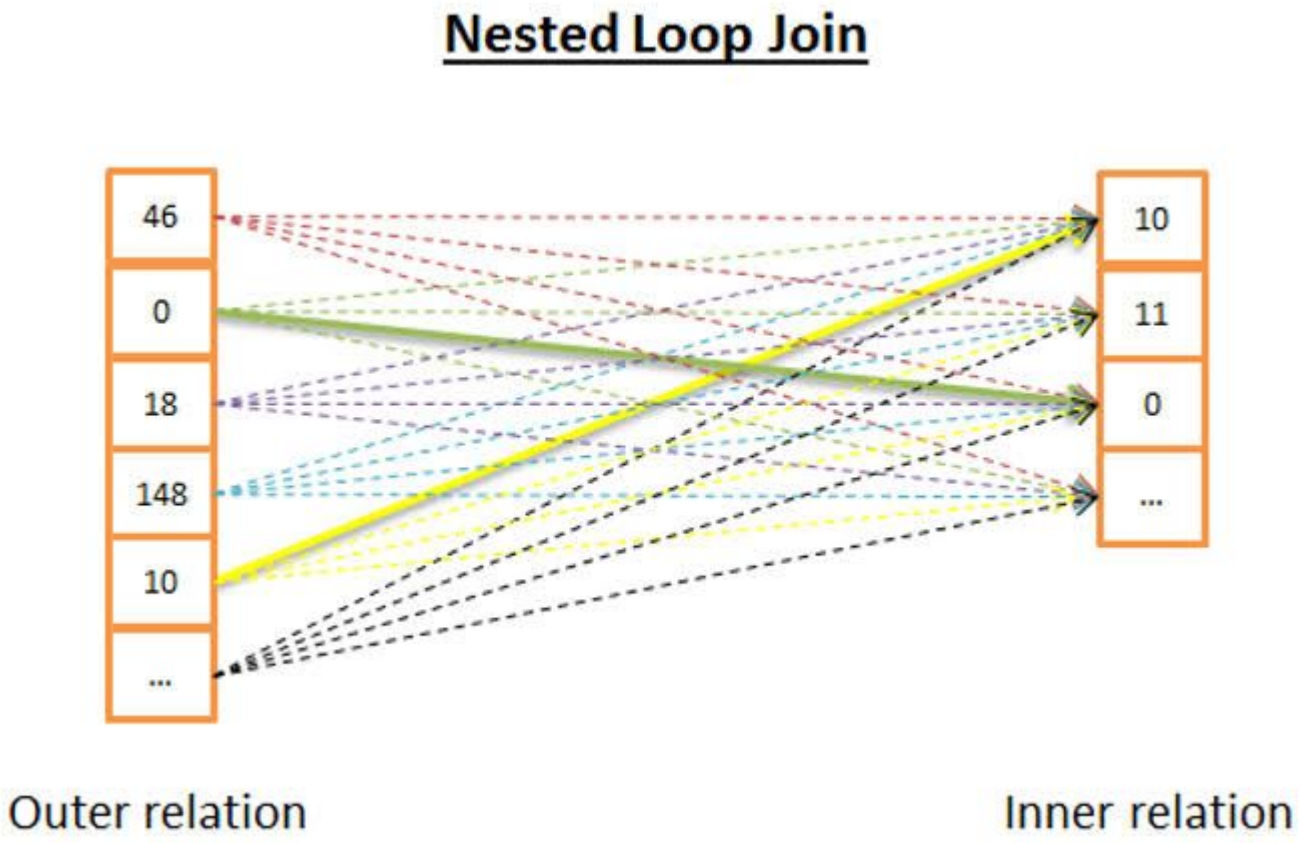
\+ works for any condition (<, ≠, LIKE …)

− far too slow for large inputs

Example: 10,000 × 10,000 rows = 10^8 comparisons.

Src: https://en.wikipedia.org/wiki/Nested_loop_join

Nested Loop Join



Img src: https://dbtut.com/index.php/2018/06/23/join-types-in-sql-server-execution-plan/

Task 2.2

SF	lineitem	orders	pairs checked
0.01	60,175	15,000	$\approx 9.0 \cdot 10^8$
0.02	120,515	30,000	$\approx 3.6 \cdot 10^9$

Task 2.3* – perf stat, per pair

```
perf stat -e cycles,instructions,LLC-loads build/tpch_join tpch/sf0_01 nested
```

per pair	SF 0.01	SF 0.02	
Instructions	≈ 6	≈ 6	same
Cycles	≈ 6	≈ 12	2×
LLC loads	≈ 0.35	≈ 1	≈ 3×

- Same code → same instructions per pair
- At SF 0.02 the order keys **no longer fit into L2** → almost every access goes to the slower LLC

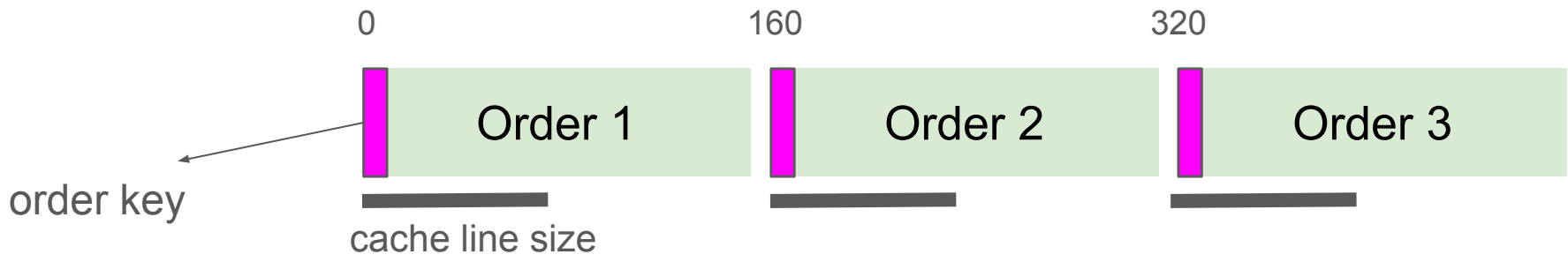
Task 2.4

0xa0 = 160

= size of an Order in bytes

To compare a **4-byte** key, the loop jumps **160 bytes**. The CPU loads whole **64-byte** cache lines → most loaded bytes are never used.

instruction	meaning
add $0xa0,%rax	move to the next order
cmp %rax,%rdx	was that the last order?
je …	if so, leave the loop
cmp (%rax),%ecx	compare the two order keys
jne …	if different, repeat



Short-circuit evaluation

Put the **cheap** check first

or the one that is **false most often**

&& and || evaluate left to right and **stop as soon as the result is known**

```
if (a && b)   // b runs only if a is true
if (a || b)   // b runs only if a is false
```

`int == int`: one instruction

`std::string == std::string`: length check + byte comparison loop

The order changes the **constant factor**, not the complexity class, but in practice still reduces time!

Src: https://en.cppreference.com/w/cpp/language/operator_logical

Task 2.5

key first

```
l.orderkey == o.orderkey && o.orderpriority == "1-URGENT"
```

string compared only when the keys match (about **once per line item)**

Runtime: several times slower – && skips its right side only when the left side is false (***short-circuit***)

priority first

```
o.orderpriority == "1-URGENT" && l.orderkey == o.orderkey
```

string compared for **every pair** (≈ $9 \cdot 10^8$ at SF 0.01)

Runtime: unchanged

Task 2.6

Build on **orders**

- Orders is the smaller table!
- **Store only urgent orders** (approximately 1/5 of all orders - ~3,000 keys at SF 0.01)

Use a Set

- Use `std::unordered_set` to store urgent order IDs
- Check if an order is urgent using a simple lookup - very fast!
- If additional order columns are needed, use `std::unordered_map`

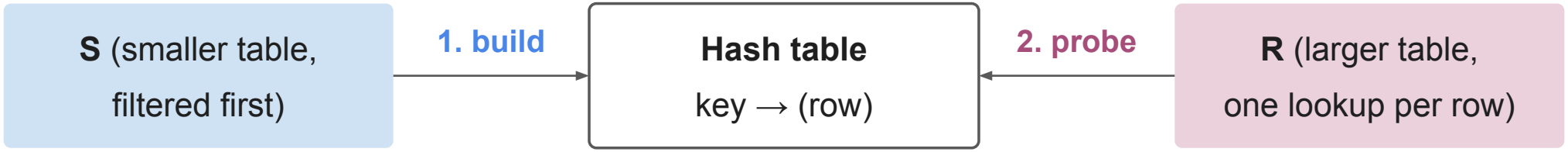
Time Complexity

- Building the set: O(|orders|)
- Checking each line item: O(1) average
- Total complexity: O(|lineitem| + |orders|)

Task 2.7

```
std::unordered_set<int> urgentOrders;
for (const Order& order : orders) {
    if (order.orderpriority == "1-URGENT") {
        urgentOrders.insert(order.orderkey);
    }
}
double sum = 0;
for (const Lineitem& lineitem : lineitems) {
    if (urgentOrders.contains(lineitem.orderkey)) {
        sum += lineitem.quantity;
    }
}
return sum;
```

Hash Join

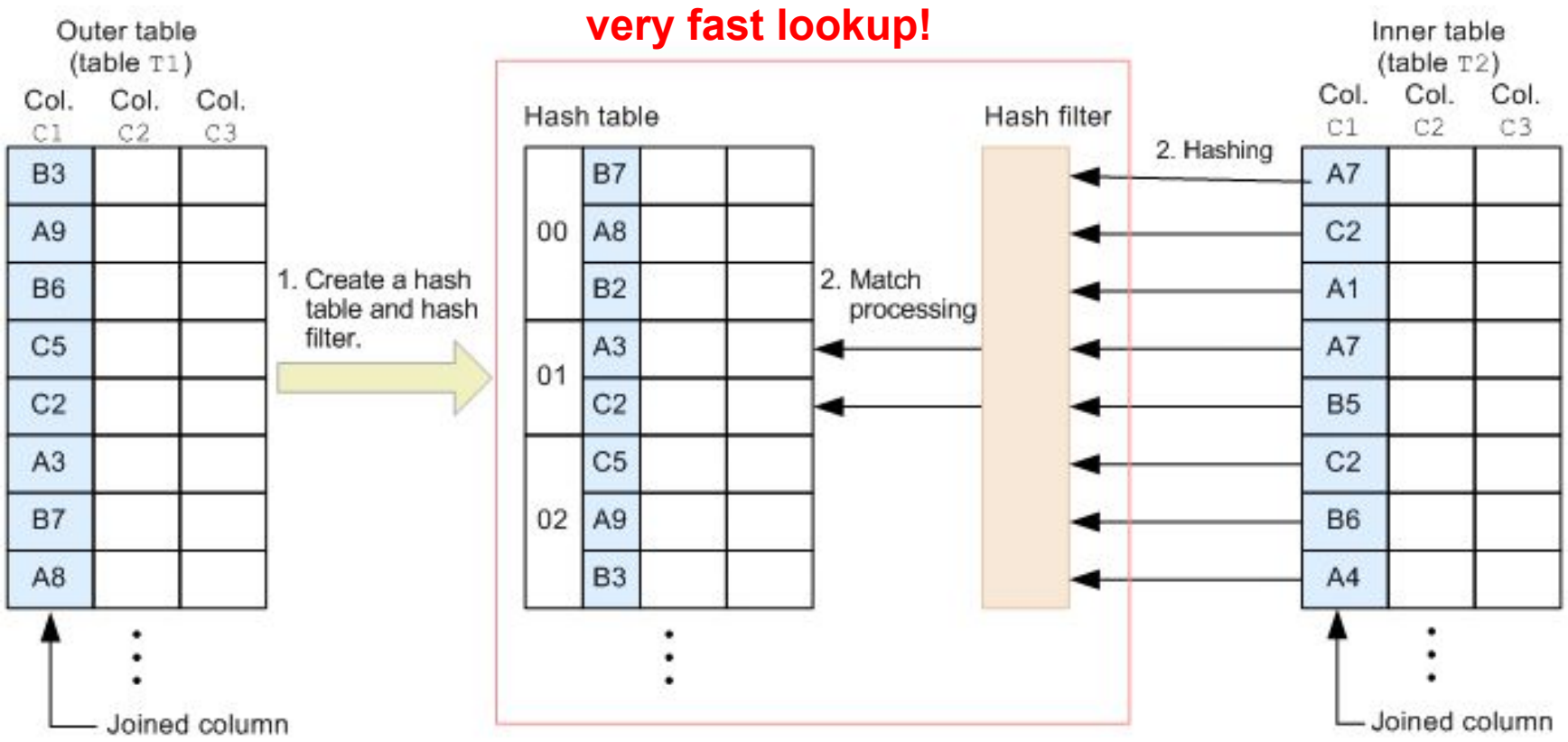


- Build: O(|S|), probe: O(1) per row on average
- Total: O(|R| + |S|) instead of O(|R|·|S|)
- **Only for equality conditions (=)**
- Fits better in caches

Hashing is very fast - O(1); basic idea: https://www.geeksforgeeks.org/dsa/introduction-to-hashing-2/

Src: https://en.wikipedia.org/wiki/Hash_join; Week 1 slides (std::unordered_map)

Hash Join



Img src: https://itpfdoc.hitachi.co.jp/manuals/3000/30006502h0e/HAPG0085.HTM

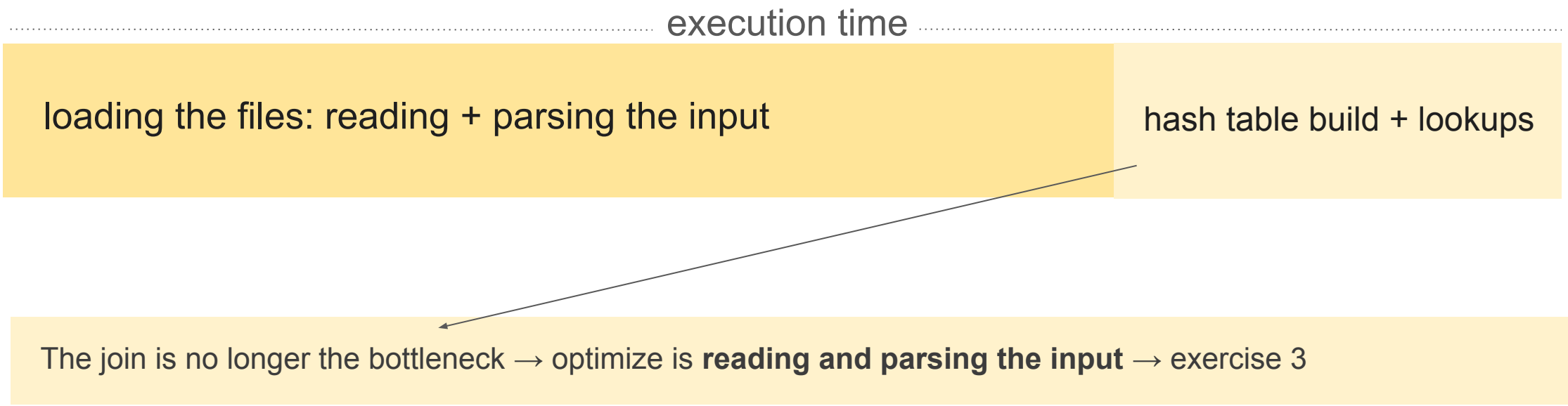
Task 2.7

Scale factor 1?

- Loading and the hash join both grow **linearly** with the data
- SF 1 = 100 × SF 0.01 → about **100 × the SF 0.01 time**
- **The nested loop would grow quadratically ≈ 10,000× – not an option!**

Task 2.8

```
perf record -g build/tpch_join tpch/sf0_02 hash
perf report
```



The join is no longer the bottleneck → optimize is **reading and parsing the input** → exercise 3

Exercise 3

Task 3.1

№	copied into	by
1	the stream's buffer	std::ifstream asks the kernel for the next block (`read` system call)
2	line	`std::getline(file, line)`
3	the string stream	`std::stringstream stream(line)`
4	field	`std::getline(stream, field, '|')`
5	the vector, then the struct	`fields.push_back(field)` ; text fields again in fromFields

all 16 columns are converted, but the query **needs only 2** (order key, quantity) – and 2 of 9 from orders.tbl.

Task 3.2

files (SF 1): **0.93 GB**

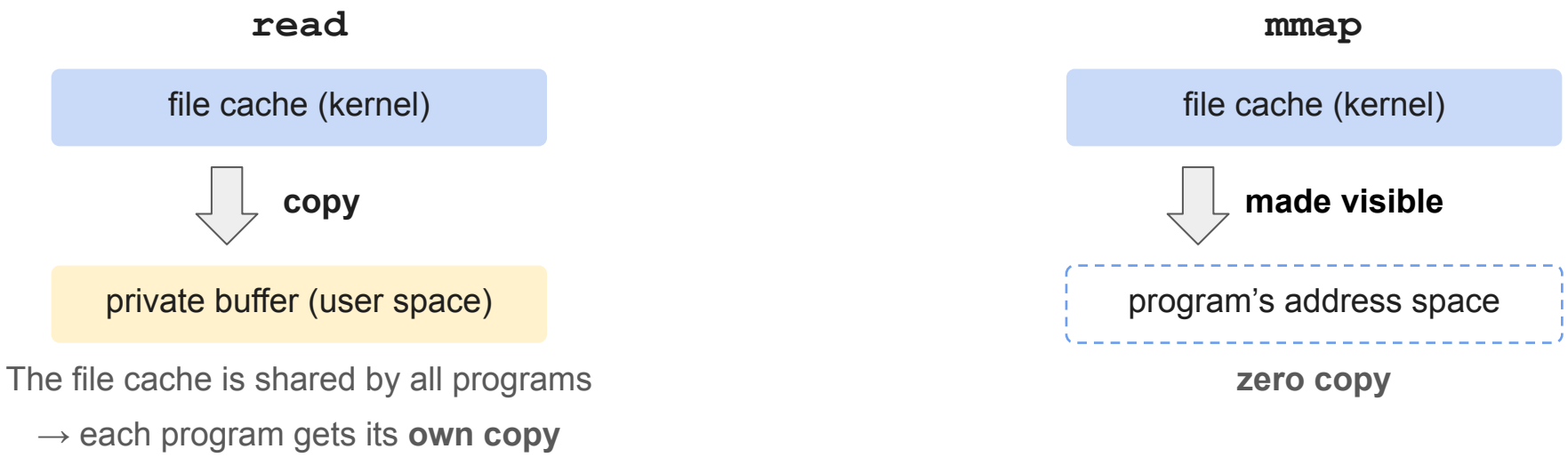
best case = **bytes / bandwidth**

source	calculation	time
file cache, ~20 GB/s	0.93 GB / 20 GB/s	~ 0.05 s
SSD, ~3 GB/s	0.93 GB / 3 GB/s	~ 0.3 s
measured, scaled up	~ 200 ms at SF 0.01 × 100	~ 20 s

scaled up is ~400× slower than (a) → the bottleneck is **not** moving the data,

but the **work the CPU does on every byte** (copies, string handling, conversion…)

Task 3.3 – read vs mmap, zero copy



If file not in cache, both must first load it from disk (very slow!).
With mmap, the first touch of an unloaded part **pauses** the program until the kernel has read it (***page fault***).

Other solutions - on Moodle